

Patrón Observer – Observable

Patrón State

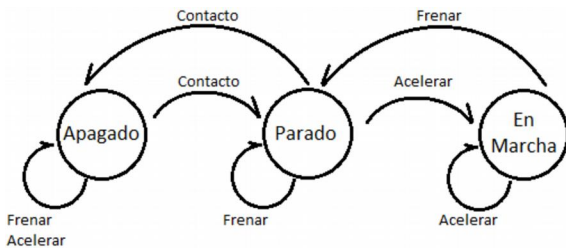
Concepto de Framework



Observable



Observer

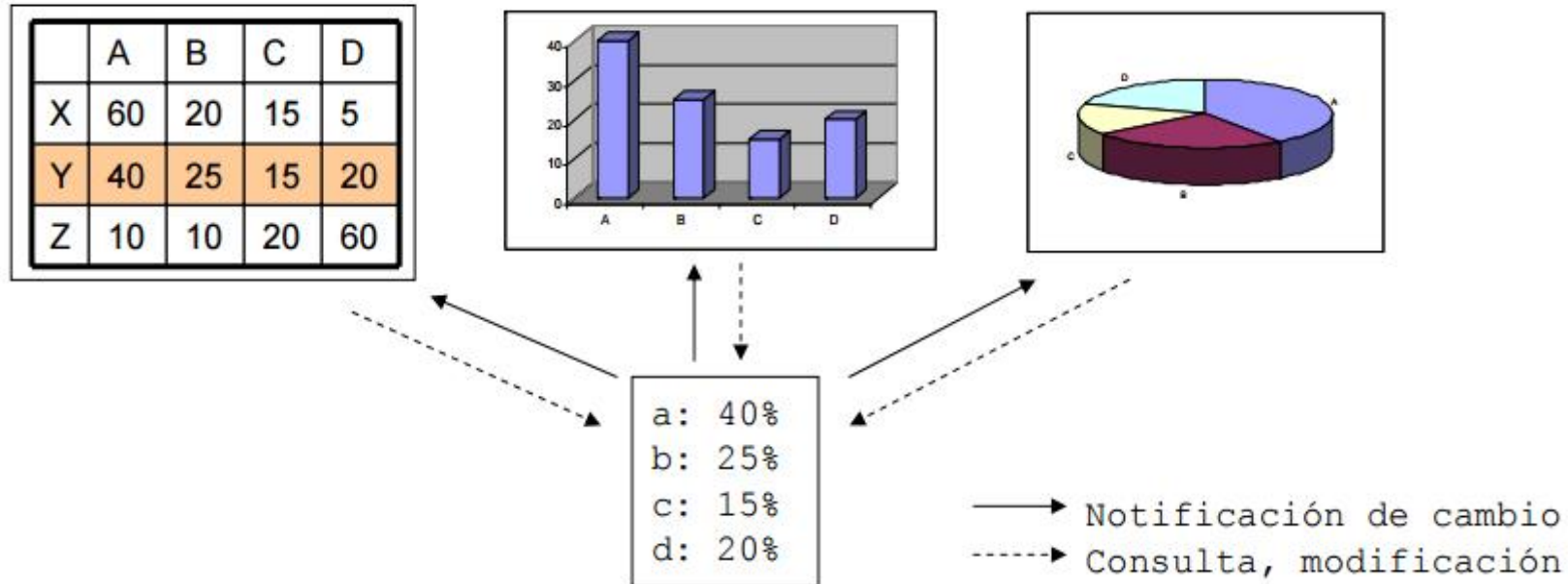


El Patrón Observer

Patrones de Diseño: El patrón OBSERVER

Intención:

Define una dependencia entre objetos de modo que, cuando un objeto cambia su estado, todos sus dependientes son notificados y actualizados automáticamente.



Patrones de Diseño: El patrón OBSERVER

Nombre alternativo:

Dependants, Publish-Subscribe.

Motivación:

Mantener la consistencia entre objetos relacionados, sin aumentar el acoplamiento entre clases

Ej: separación de la capa de presentación en una interfaz de usuario de los datos de aplicación subyacentes

Patrones de Diseño: El patrón OBSERVER

Aplicabilidad:

Usar este patrón en cualquier de estas situaciones:

- Cuando una abstracción tiene dos aspectos, uno dependiente del otro.
- Cuando un cambio en un objeto requiere que se actualicen otros, sin saber cuantos objetos hay que actualizar.
- Cuando un objeto debería ser capaz de notificar a otros objetos sobre su cambio, pero sin hacer presunciones sobre quienes son esos objetos.

Patrones de Diseño: El patrón OBSERVER

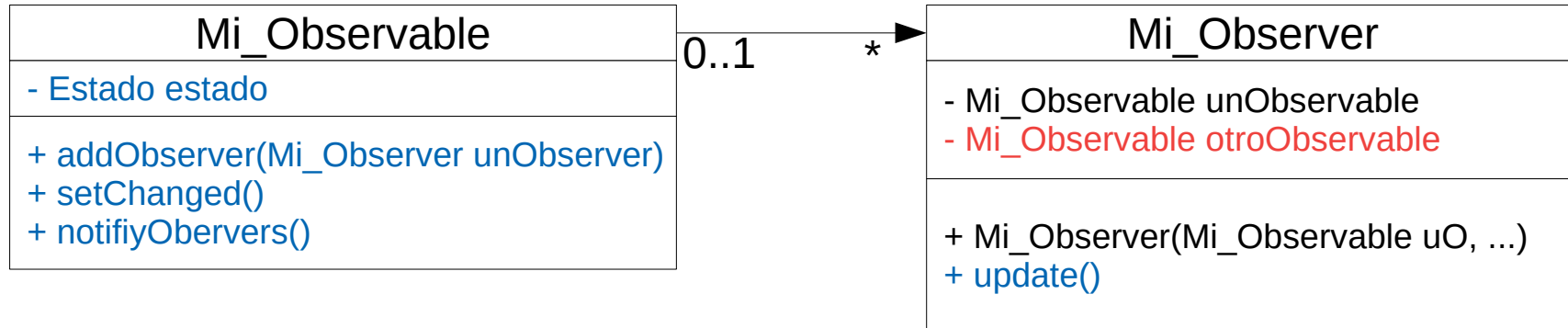
Bosquejo de clases



Observable



Observer



Hereda de clase abstracta

Implementa interfaz

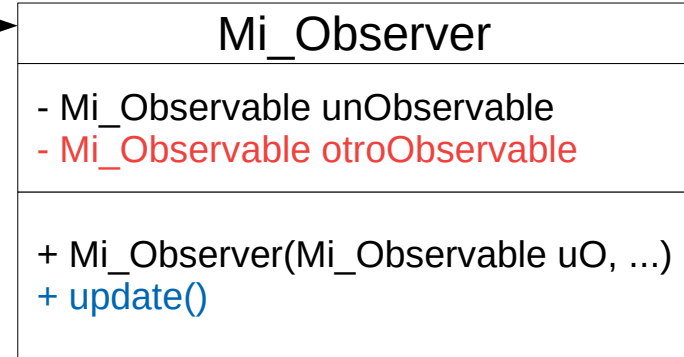
Patrones de Diseño: El patrón OBSERVER



Observable



0..1 *

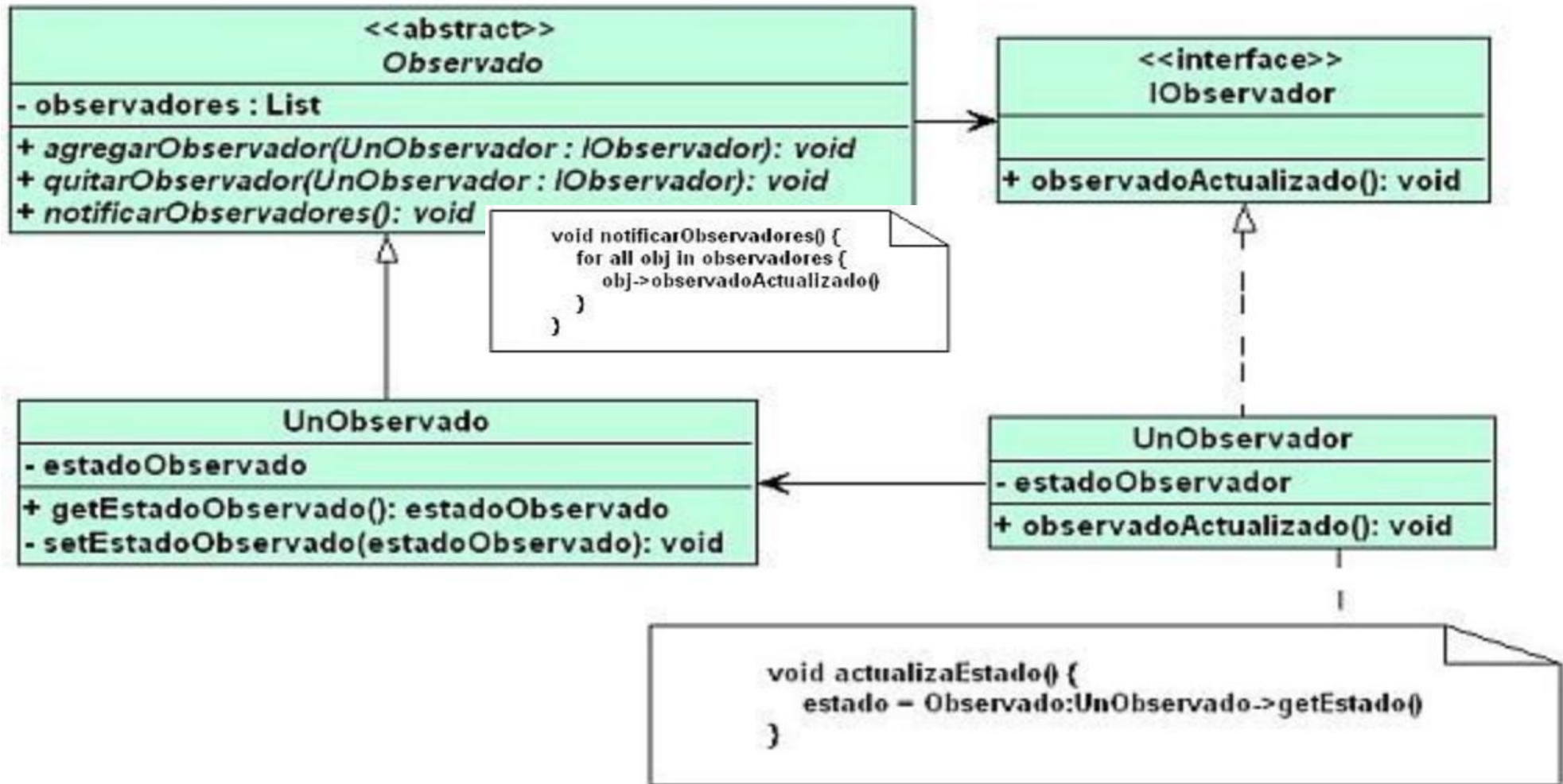


Observer

El observable (heredando de una clase abstracta) mantiene una colección de los observadores. Cada vez que sucede algo significativo para este objeto emite una notificación a todos los observadores. Esto provocará que se ejecute un método en cada observador

El observador mantiene una referencia de cada objeto observado (puede ser más de uno). Implementa el método update (propio de la interfaz que implementa) que se ejecuta cuando los observados notifican. Es responsabilidad del observador mantener vincularse con el observado.

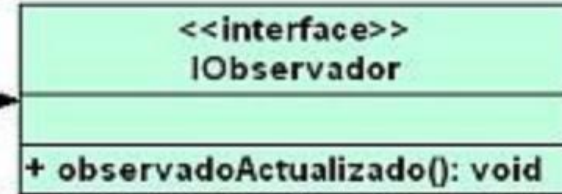
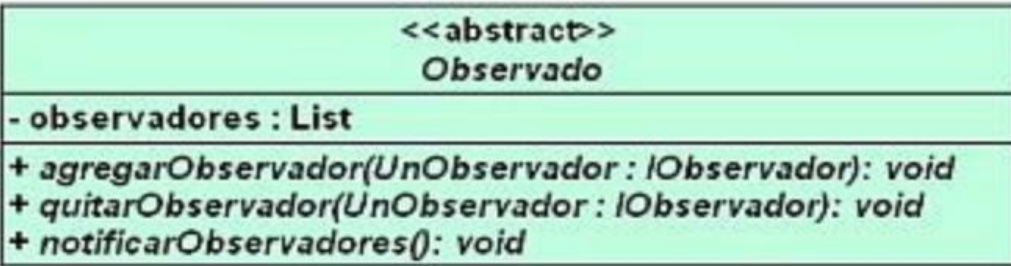
Patrones de Diseño: El patrón OBSERVER



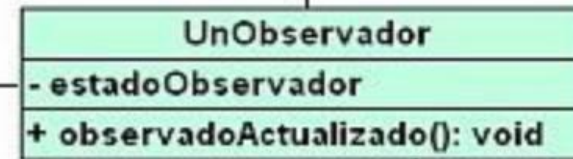
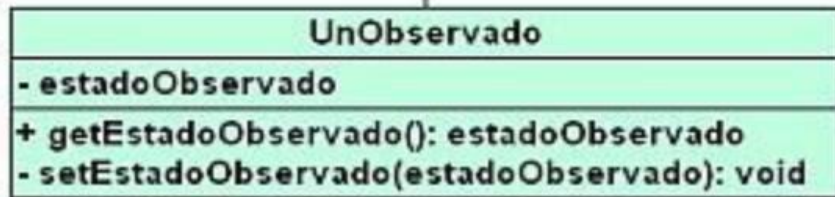
Patrones de Diseño: El patrón OBSERVER

- conoce a sus observadores, que pueden ser un número arbitrario
- proporciona una interfaz para añadir y quitar objetos observadores

- define la interfaz de los objetos a los que se deben notificar cambios en un Observado



- mantiene una referencia a unObservado
- almacena el estado del Observado que le resulta de interés
- implementa la interfaz de Observer para mantener su estado consistente con el del Observado



- almacena el estado de interés para sus observadores
- envía notificaciones a sus observadores cuando su estado cambia

```
void actualizaEstado() {
    estado = Observado:UnObservado->getEstado()
}
```

Patrones de Diseño: El patrón OBSERVER

Participantes

Observado <abstracta>:

- conoce a sus observadores, que pueden ser un número arbitrario
- proporciona una interfaz para añadir y quitar objetos observadores

Observer (u Observador) <interface>:

- define la interfaz de los objetos a los que se deben notificar cambios en un sujeto

UnObservado <concreto>:

- almacena el estado de interés para sus observadores
- envía notificaciones a sus observadores cuando su estado cambia

UnObservador <concreto>:

- mantiene una referencia a unObservado
- almacena el estado del observado que le resulta de interés
- implementa la interfaz de observer para mantener su estado consistente con el del sujeto

Patrones de Diseño: El patrón OBSERVER

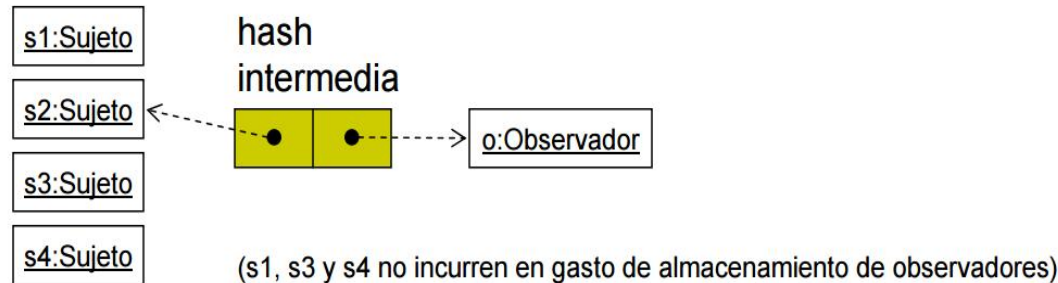
Consecuencias:

- Permite modificar observados y observadores de manera independiente
- Permite reutilizar un sujeto sin reutilizar sus observadores, y viceversa
- Permite añadir observadores sin tener que cambiar el observado ni los demás observadores
- Acoplamiento abstracto entre el observado y el observador. El sujeto no sabe la clase concreta de sus observadores (acoplamiento mínimo).
- Soporte para broadcast. El sujeto envía la notificación a todos los observadores suscritos. Se pueden añadir/quitar observadores.
- Actualizaciones inesperadas. Una operación en el observado puede desencadenar una cascada de cambios en sus observadores. El protocolo no ofrece detalles sobre lo que ha cambiado.

Patrones de Diseño: El patrón OBSERVER

Implementación:

- Correspondencia entre observados y observadores
 - Usualmente, el sujeto guarda una referencia a sus observadores
 - Costoso si hay muchos observados y pocos observadores. En ese caso:
 - usar tabla hash: menor coste en espacio, pero mayor coste en tiempo



- Observar más de un observado (ej. hoja de cálculo con 2 fuentes de datos)
- Extender la interfaz de actualización para que el observador sepa qué sujeto cambió de estado (por ej. pasar el sujeto en la llamada a update).

Patrones de Diseño: El patrón OBSERVER

Implementación:

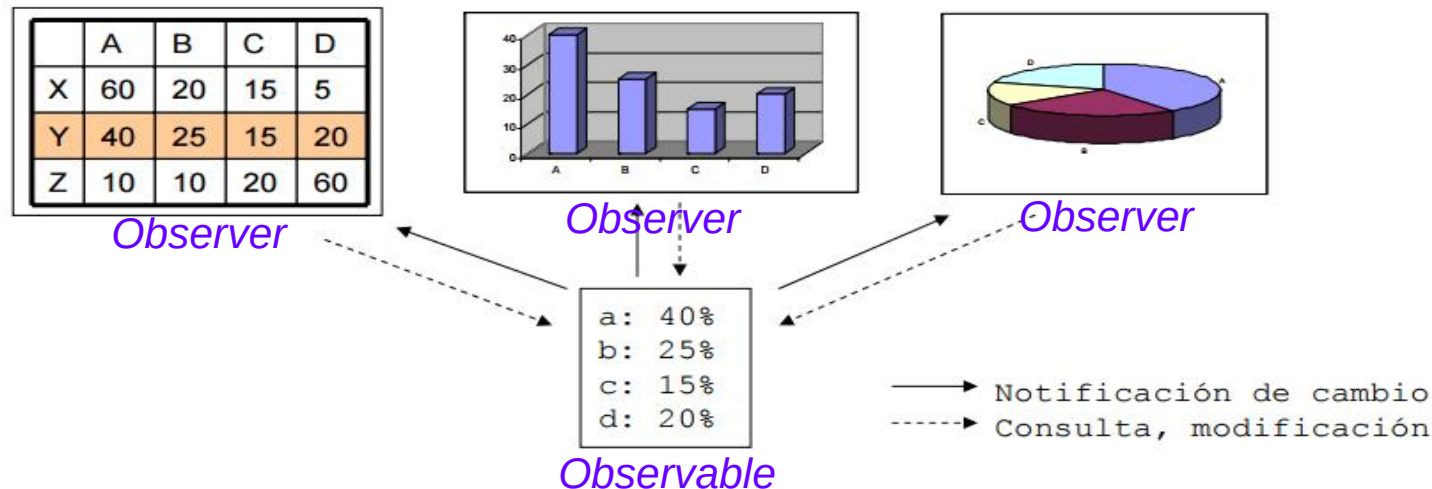
- ¿Quién dispara la actualización llamando a notify?
 - El sujeto desde aquellos métodos que cambian su estado
 - ventaja: las clases cliente no tienen que hacer nada
 - inconveniente: no es óptimo si hay varios cambios de estado seguidos
 - Las clases cliente
 - ventaja: se puede optimizar llamando a notify tras varios cambios
 - inconveniente: los clientes tienen la responsabilidad de llamar a notify
- Referencias perdidas a sujetos que se han eliminado
 - Se puede evitar notificando la eliminación del sujeto a sus observadores
 - En general, eliminar los observadores del sujeto eliminado no es una opción

Java: OBSERVER – OBSERVABLE



La interface Observer y la clase Observable

Proporcionan un protocolo para que un número arbitrario de objetos **Observer** vigilen cambios y eventos en un número cualquiera de objetos de tipo **Observable**.



Java: OBSERVER – OBSERVABLE

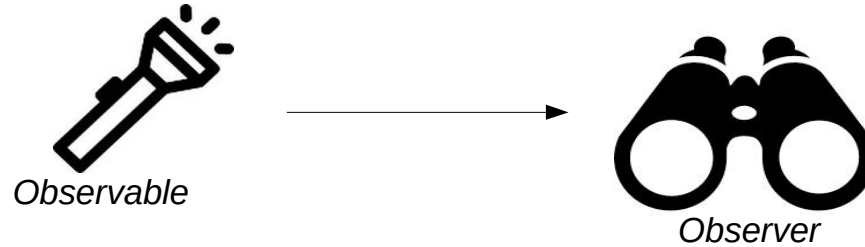


Los objetos Observable proporcionan métodos para mantener una lista de objetos Observer, que desean saber cambios que se pueden producir en el objeto Observable.

Los objetos Observable extienden la clase Observable.
Los objetos Observer implementan la interfaz Observer.



Java: OBSERVER – OBSERVABLE



```
setChanged();  
notifyObservers(...);
```

```
update(...)
```

Para informar sobre un cambio, el objeto Observable invoca al método `notifyObservers`, que a su vez invoca al método `update` de cada objeto Observer.

interfaz Observer



update(...)

Consta de un solo método:

```
public void update( Observable obj, Object param)
```

El método es invocado cuando el objeto **Observable obj** tiene un cambio que informar. El parámetro **param** es una forma de pasar un objeto arbitrario que describa el cambio.

interfaz Observer



```
public void update(Observable obj, Object param)
```

obj

Referencia al objeto que ejecuta
notifyObservers(...)

param

Objeto que describe el cambio que se quiere
notificar

Clase Observable



Observable

Cada objeto Observable mantiene un **indicador de “modificado”** que es utilizado por los métodos de la subclase para indicar que ha sucedido algo de interés.

```
setChanged();  
notifyObservers(...);
```

Cuando ocurre un cambio, el objeto Observable debe invocar al método **setChanged** y después notificarlo a los observadores.

Clase Observable



Observable

protected void setChanged()

- Marca que el objeto actual ha sido modificado.

protected void clearChanged()

- Indica que el objeto actual no ha cambiado.

public boolean hasChanged()

- Devuelve el valor actual de indicador de “modificado”

Clase Observable



Observable

```
public void notifyObserver(Object  
param)
```

- Notifica a todos los objetos Observer de la lista que algo ha ocurrido, y después borra el indicador de “modificado”. Para cada método observador de la lista, se invoca a su método update con el objeto Observable actual como primer parámetro y param como segundo parámetro.

Clase Observable



Observable

public void notifyObservers()

- Equivale al anterior con parámetro null

public void addObserver(Observer o)

- Agrega un observador a la lista

public void deleteObserver(Observer o)

- Borra el observador o de la lista

public deleteObservers()

- Borra todos los observadores de la lista

public int countObservers()

- Devuelve el número de observadores que hay en la lista

Ejemplo



Observable

```
public class Usuarios extends Observable
{
    private Map conectados = new HashMap();

    public void login(String nombre, String contraseña)
        throws UsuarioIncorrectoException
    {
        if (!contraseñaValida(nombre, contraseña))
        {
            throw new UsuarioIncorrectoException(nombre);
        }
        EstadoUsuario estado = new EstadoUsuario(nombre);
        conectados.put(nombre, estado);
        setChanged();
        notifyObservers(estado);
    }

    public void logout(EstadoUsuario estado)
    {
        conectados.remove(estado.nombre());
        setChanged();
        notifyObservers(estado);
    } // .....
}
```



Ejemplo

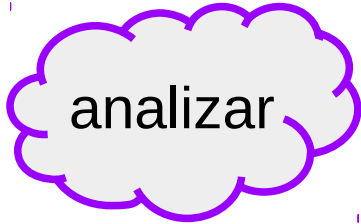


```
public class Ojo implements Observer
{
    Usuarios observado;

    public Ojo(Usuarios usuario)
    {
        observado = usuario;
        observado.addObserver(this);
    }

    public void update(Observable usuario, Object queEstado)
    {
        if (usuario != observado)
        {
            throw new IllegalArgumentException();
        }
        EstadoUsuario estado = (EstadoUsuario) queEstado;
        if (observado.conectados(estado))
            añadirUsuario(estado);
        else
            borrarUsuario(estado);
    }
}
```

Ejemplo



```
public class Ojo implements Observer
{
    Usuarios observado;

    public Ojo(Usuarios usuario)
    {
        observado = usuario;
        observado.addObserver(this);
    }

    public void update(Observable usuario, Object queEstado)
    {
        if (usuario != observado)
        {
            throw new IllegalArgumentException();
        }
        EstadoUsuario estado = (EstadoUsuario) queEstado;
        if (observado.conectados(estado))
            añadirUsuario(estado);
        else
            borrarUsuario(estado);
    }
}
```

Ejemplo

Cada objeto ojo observa a un objeto Usuarios particular. Cuando un usuario se conecta o se desconecta, *Ojo* es informado porque invocó al método **addObserver** del objeto Usuarios con sí mismo como objeto interesado. Cuando se invoca **update**, comprueba la corrección de sus parámetros y después se modifica la presentación dependiendo de si el usuario se ha conectado o se ha desconectado.

```
public class Ojo implements Observer
{
    Usuarios observado;

    public Ojo(Usuarios usuario)
    {
        observado = usuario;
        observado.addObserver(this);
    }

    public void update(Observable usuario, Object queEstado)
    {
        if (usuario != observado)
        {
            throw new IllegalArgumentException();
        }
        EstadoUsuario estado = (EstadoUsuario) queEstado;
        if (observado.conectados(estado))
            añadirUsuario(estado);
        else
            borrarUsuario(estado);
    }
}
```



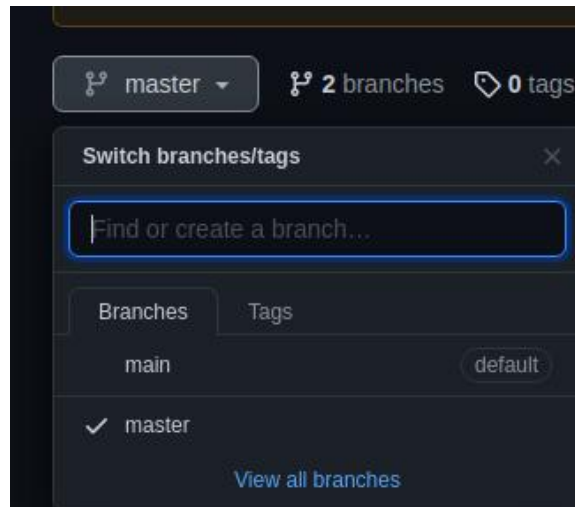
Otro ejemplo



Ver código del proyecto Ejemplo3 dentro de la aplicación Jdeveloper
ObserverObservable.

Bajarlo del repositorio https://github.com/leonelguccione/2021_ObserverObservable

Rama master



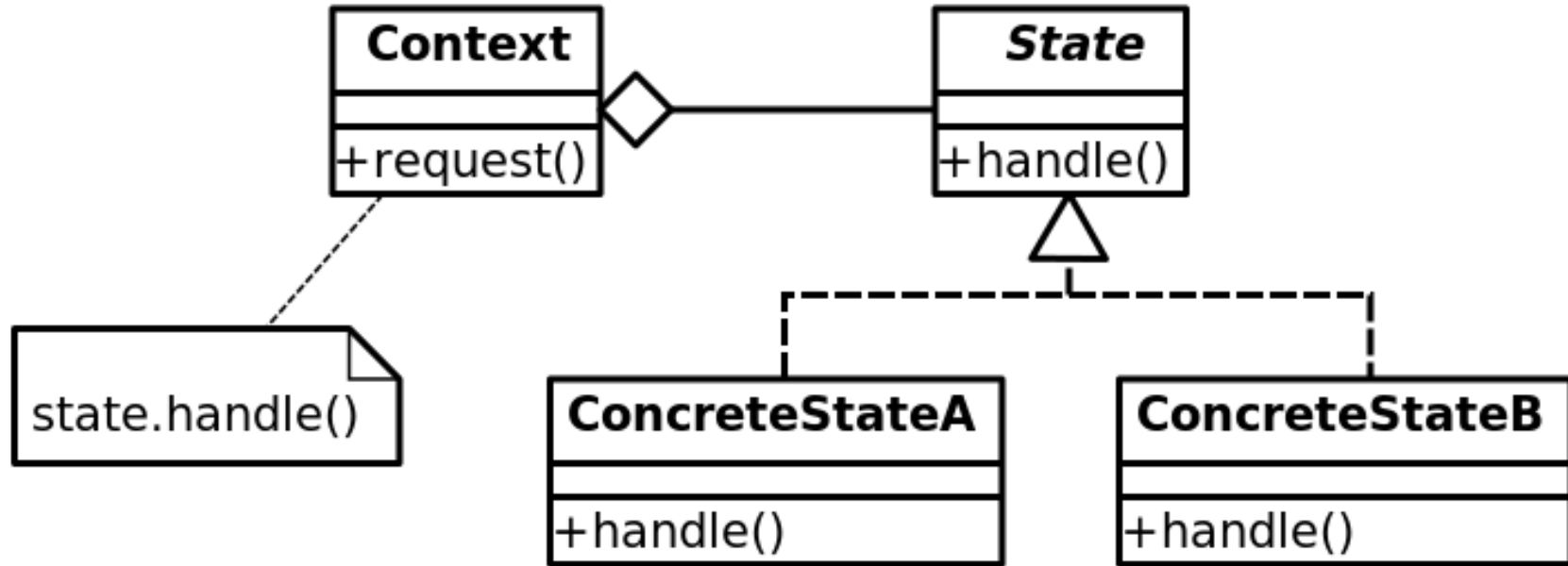
El Patrón State

(de comportamiento)

El Patrón State

Objetivo:

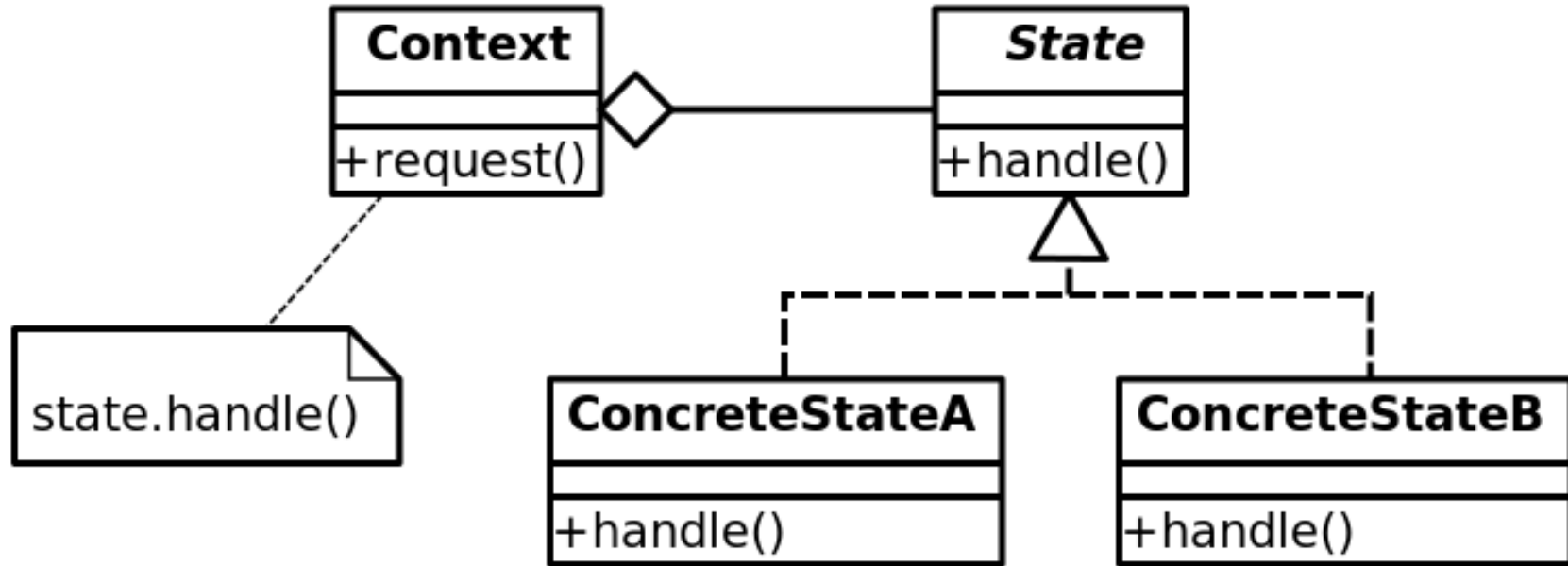
Permitir que un objeto modifique su comportamiento cuando su estado interno cambie. Parecerá que el objeto cambia de clase.



El Patrón State

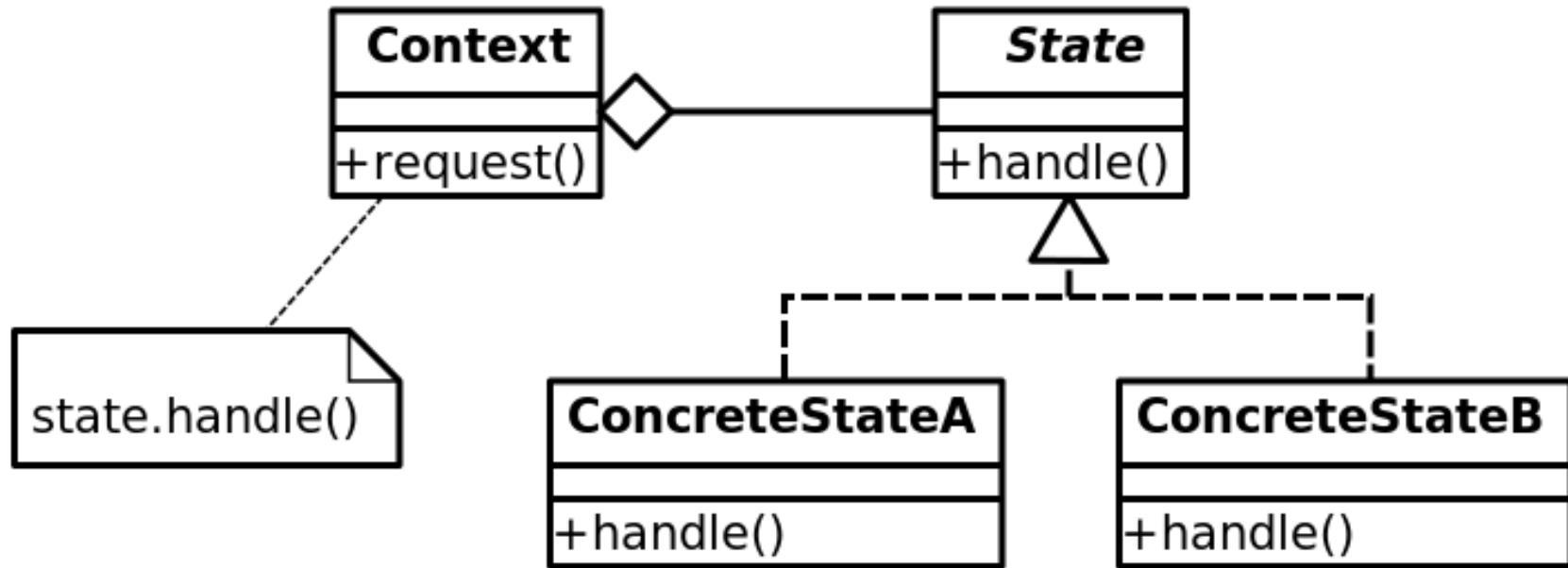
Objetivo:

El patrón State tiene la misión fundamental de encapsular el comportamiento de un objeto dependiendo del estado en el que éste se encuentre.



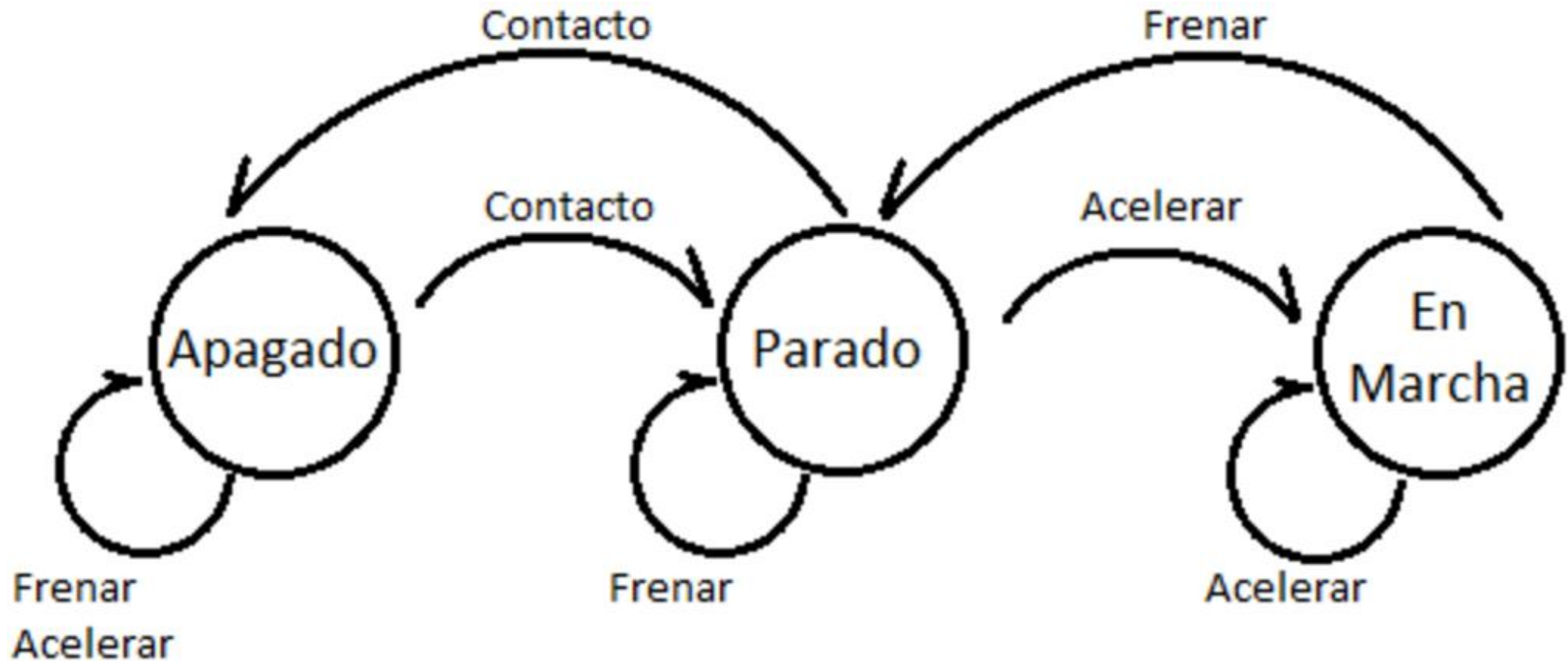
El Patrón State

¿A qué nos referimos con estado? Estrictamente hablando, podemos definir **el estado de un objeto como el conjunto actual de los valores de los atributos de un objeto.**



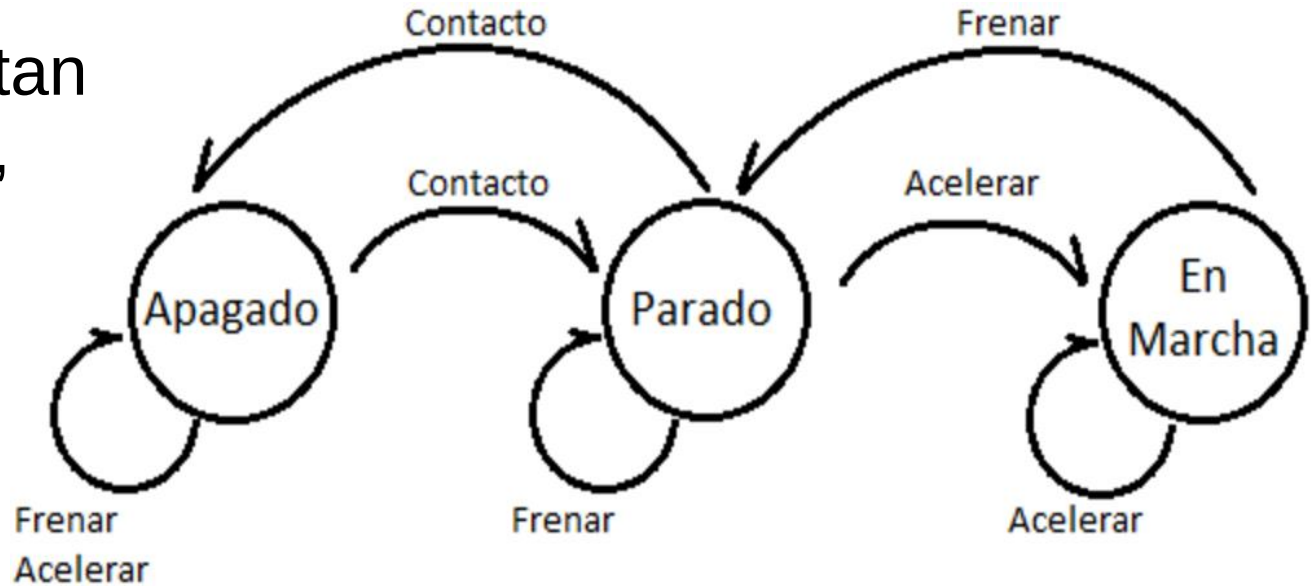
El Patrón State - Ejemplo

Estados de un vehículo



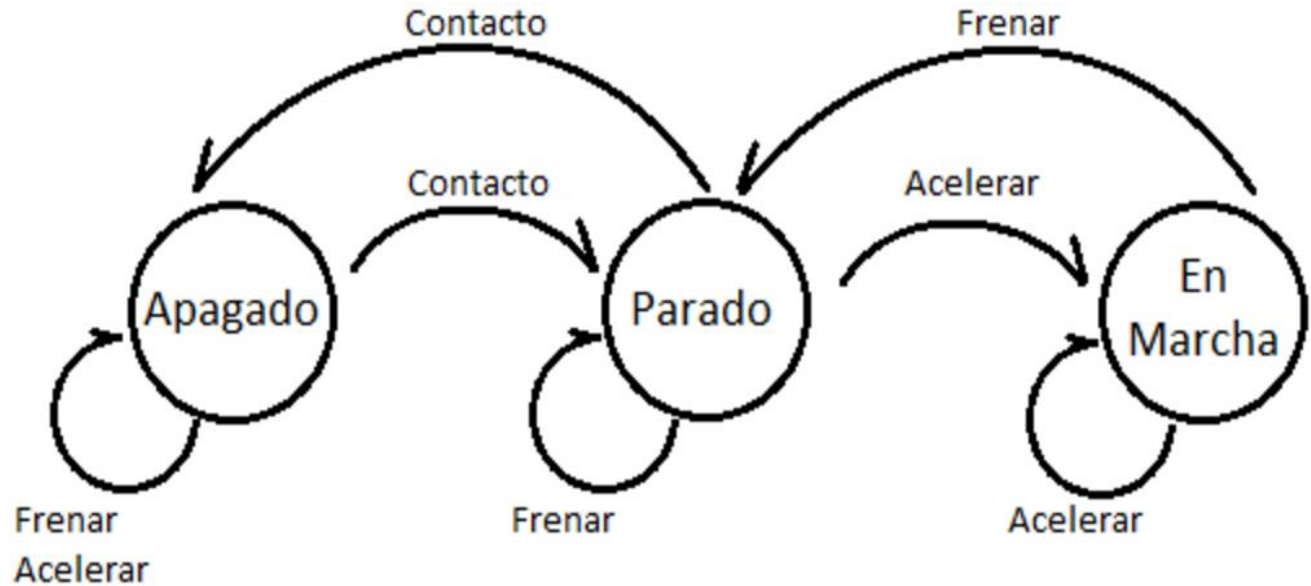
El Patrón State - Ejemplo

Los **círculos** representan los **estados**: Apagado, Parado, En Marcha.



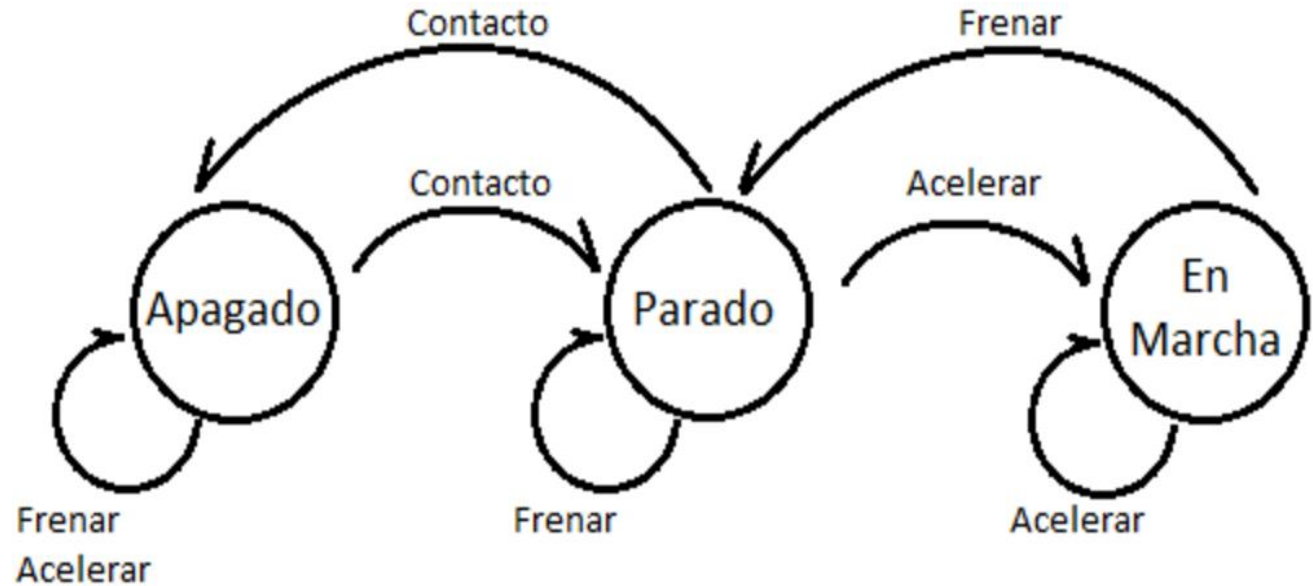
Cada uno de esos estados se caracteriza por los valores de una serie de atributos (por ejemplo, para que el vehículo se encuentre En Marcha, se debe dar el caso de que el vehículo esté arrancado y con una velocidad distinta de cero).

El Patrón State - Ejemplo



¿Cómo se producen los cambios de estado? A través de las llamadas **transiciones**. Estas transiciones podrían compararse con nuestros **métodos**, que actúan sobre los valores del objeto haciendo que éste cambie de estado. O no.

El Patrón State - Ejemplo

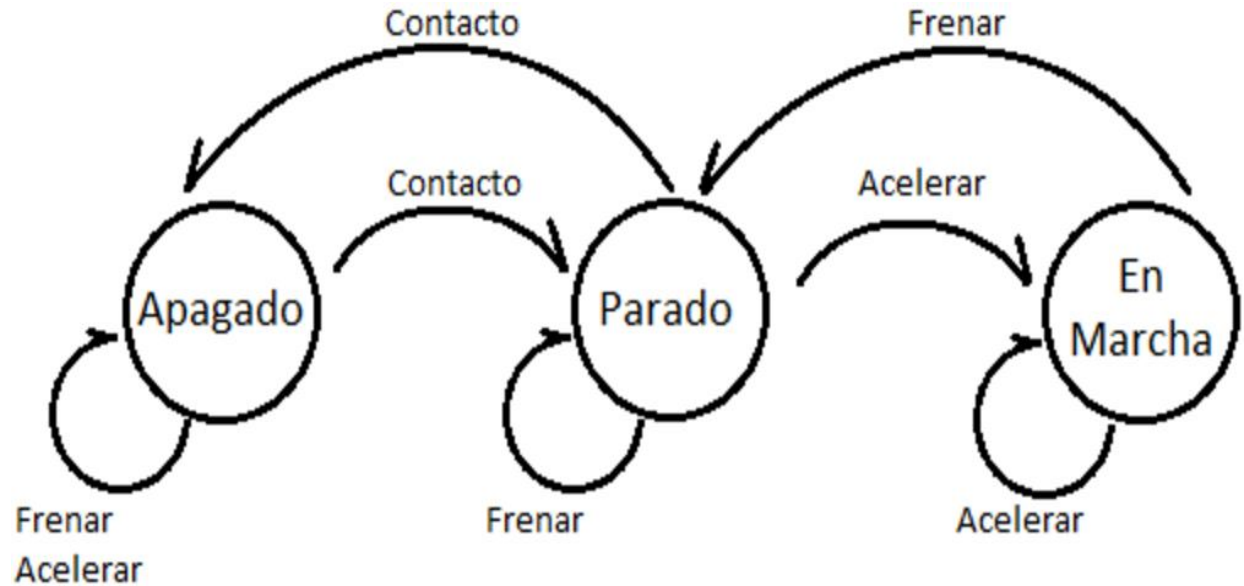


El factor determinante se encuentra en que **dependiendo en qué estado nos encontremos, las transiciones actuarán de forma distinta**, realizando ciertas operaciones, transiciones a otros estados... o no realizando absolutamente nada.

El Patrón State - Ejemplo

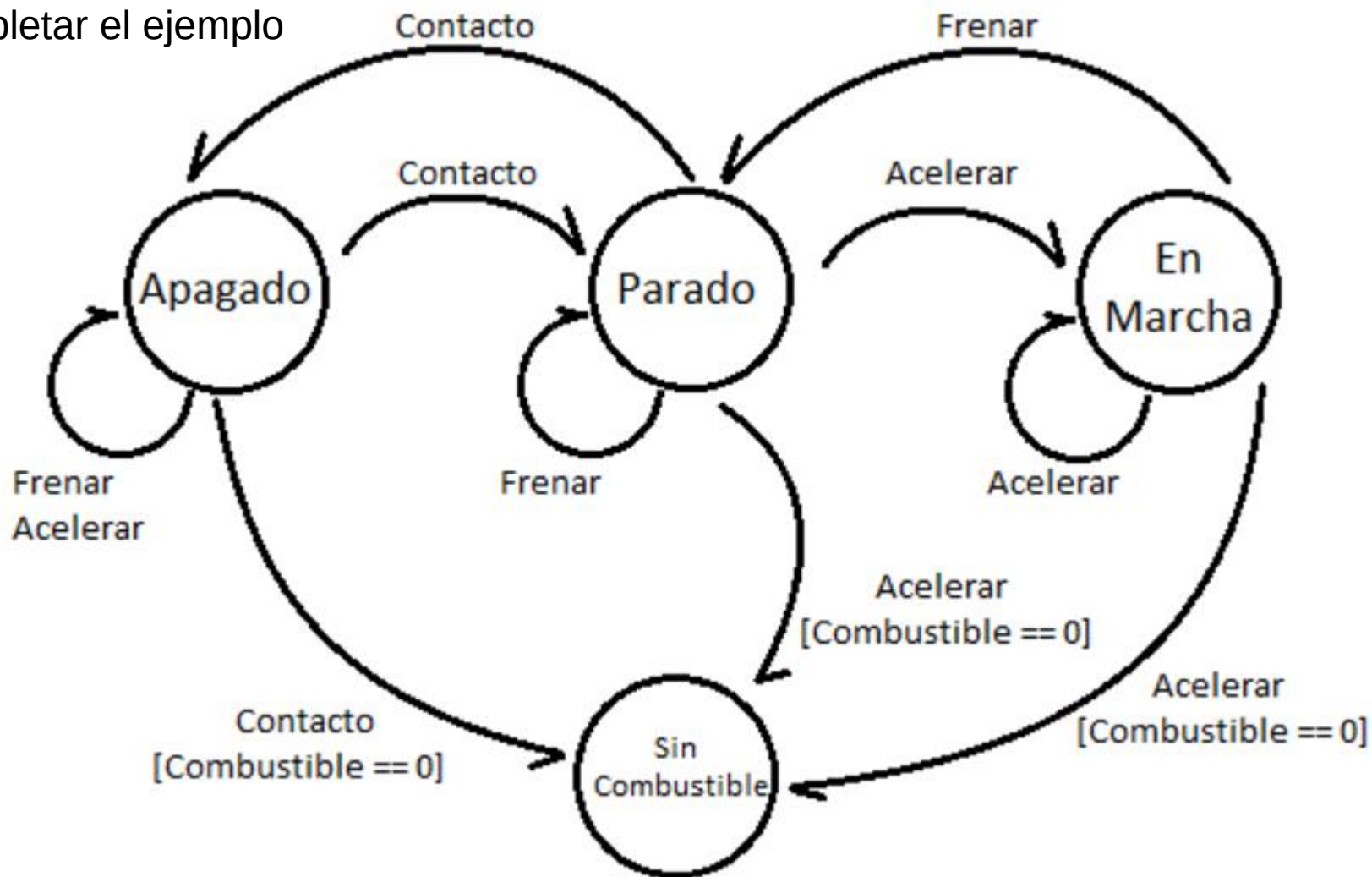
Problema:

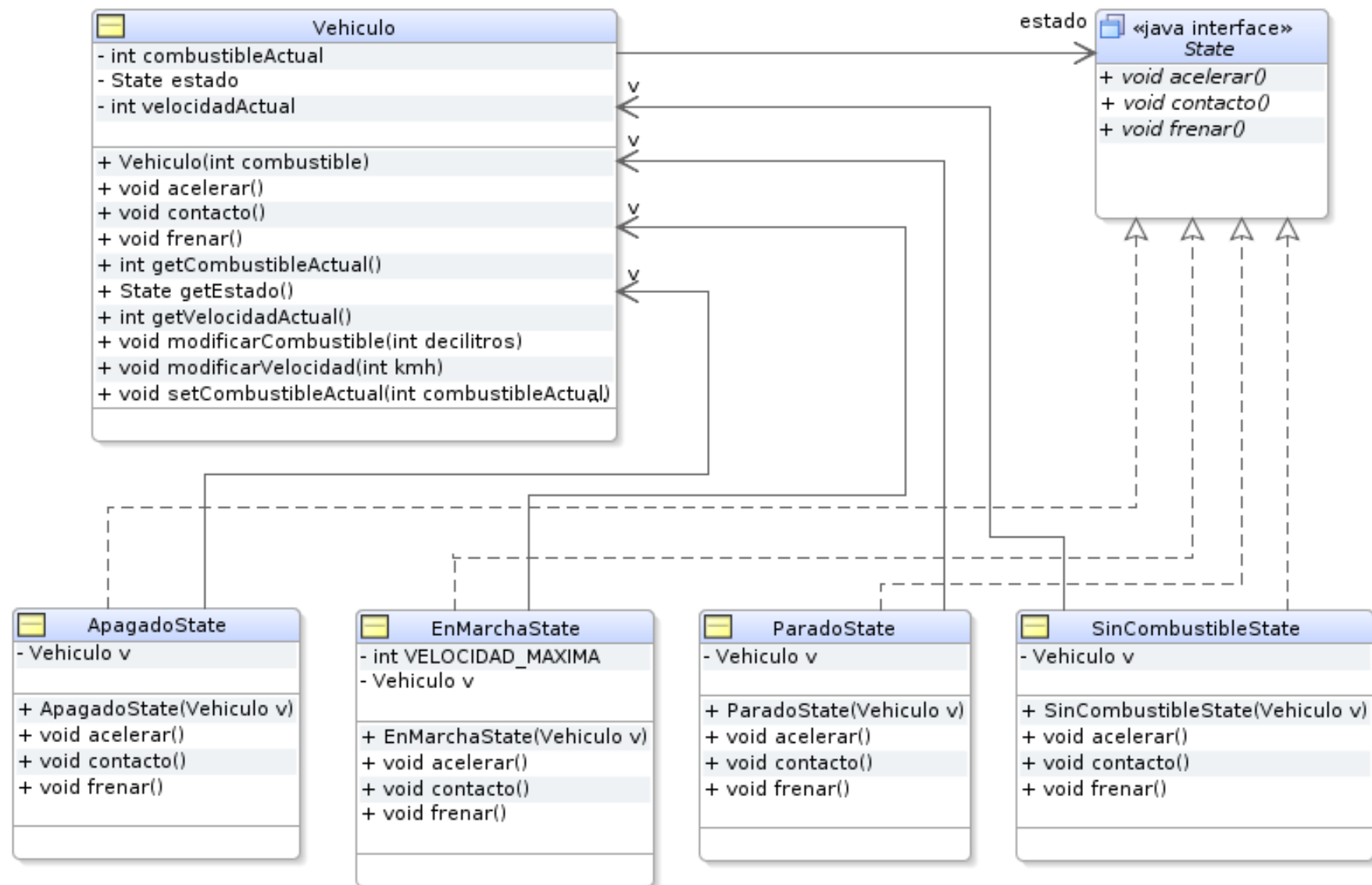
Cada método debería realizar algo diferente según sea el estado del objeto. El problema es usar “if” para preguntar por el estado y seleccionar el código correcto.



Solución: encapsular el estado.

Agregamos un nuevo estado para completar el ejemplo





El Patrón State - Ejemplo



Lo primero que deberemos hacer será crear una interfaz común que todos los estados deberán implementar. Esta interfaz, por lo tanto, deberá exponer los métodos Acelerar, Frenar y Contacto.

```
public interface State
{
    void acelerar();

    .....
    void frenar();

    .....
    void contacto();
}
```

El Patrón State - Ejemplo



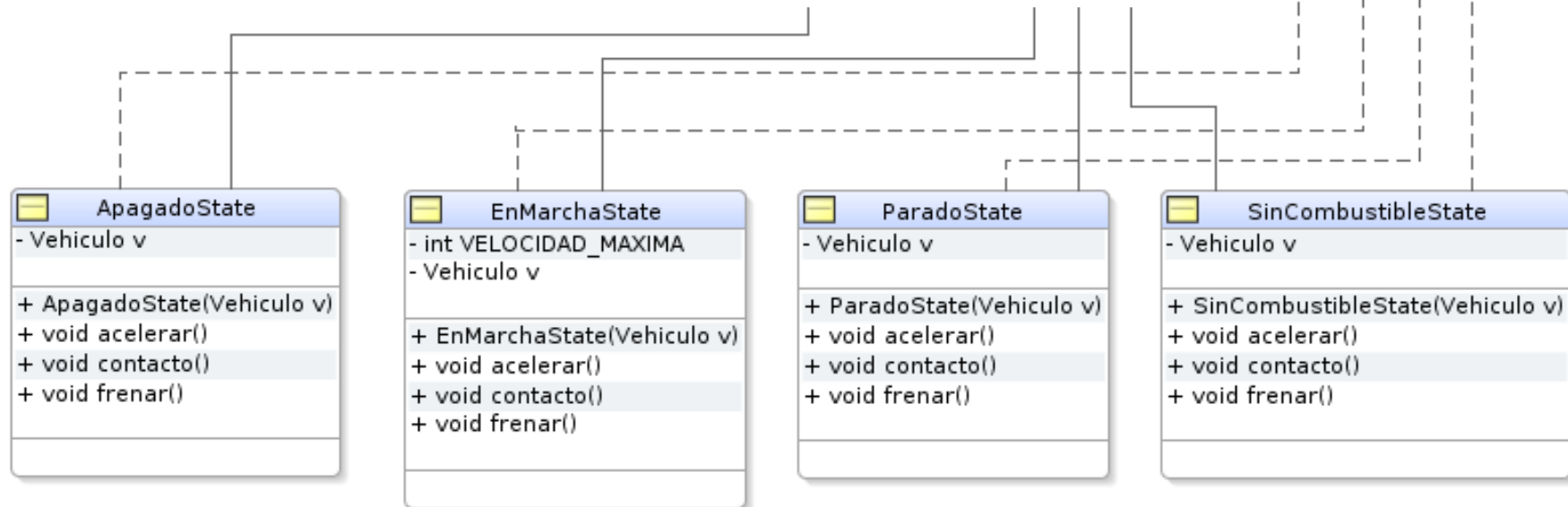
Lo siguiente que haremos será crear una clase que se denominará clase de contexto, encargada de albergar el estado actual junto al resto de los atributos que se consideren oportunos. En nuestro caso concreto, será el vehículo:

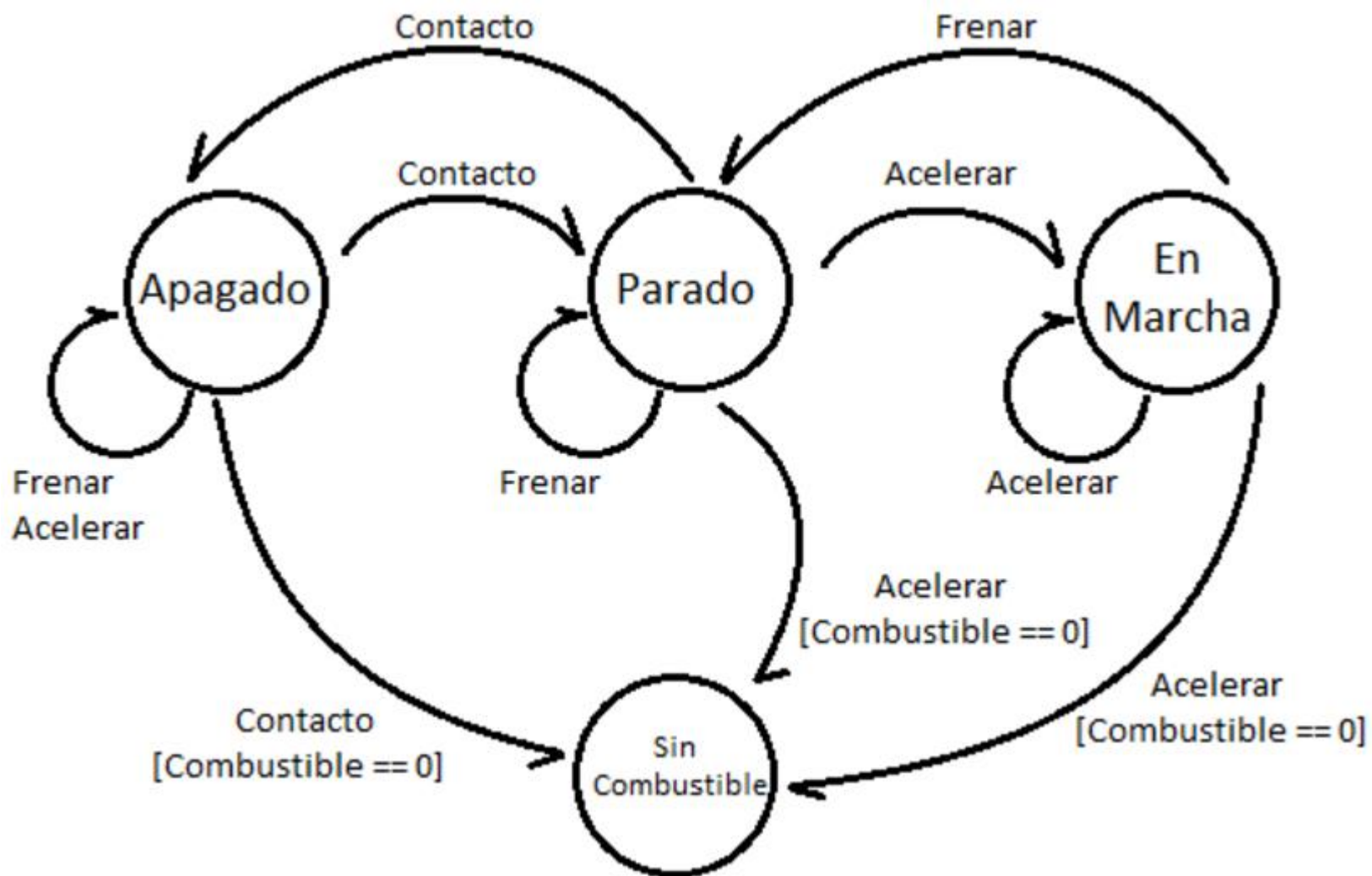
```
public class Vehiculo
{
    private State estado; // Estado actual del vehiculo (apagado,
                          // parado, en marcha, sin combustible)
    private int velocidadActual = 0; // Velocidad actual del vehiculo
    private int combustibleActual = 0; // Cantidad de combustible restante

    // El constructor inserta el combustible del que dispondra el vehiculo
    public Vehiculo(int combustible)
    {
        this.combustibleActual = combustible;
        //Indicar un estado inicial (Apagado)
        estado = new ApagadoState(this);
    }
}
```



Antes se debe implementar la interfaz con una clase por cada estado existente en el sistema. La idea será crear una clase por cada uno de los estados, que implemente los métodos de la interfaz State. Además, añadiremos un nuevo estado **Sin Combustible**, que será un estado final (una vez alcanzado, no podrá cambiarse de estado).

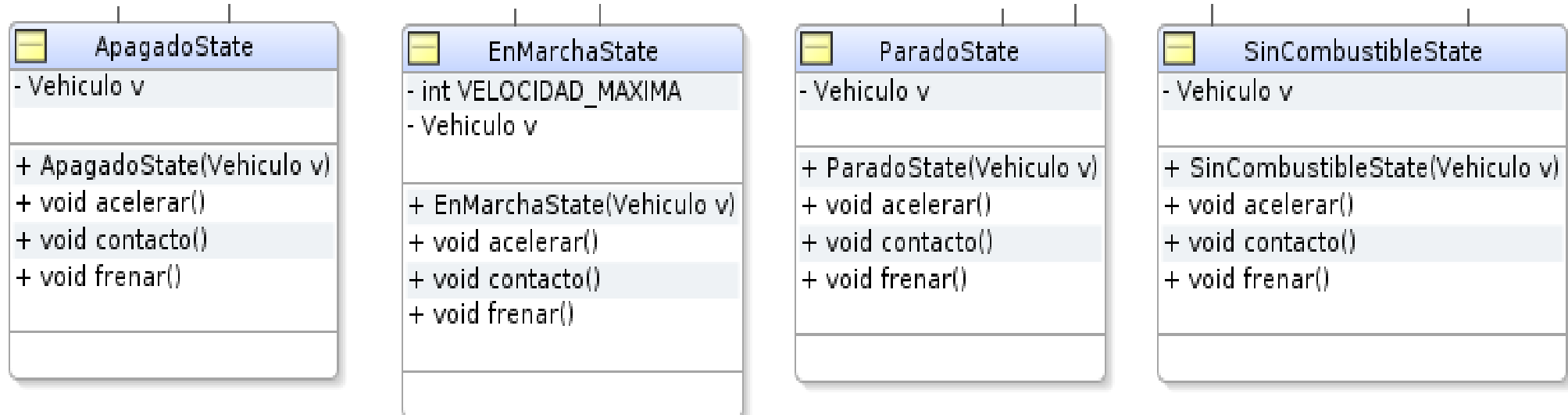




El Patrón State - Ejemplo



Las cuatro clases que implementan los estados tienen como característica común (además de los métodos de la interfaz) que su constructor recibe como parámetro una referencia al contexto, de modo que puedan utilizar sus métodos públicos y, sobre todo, para que puedan realizar la transición de estados inyectando ese objeto en los nuevos objetos que se creen.



El Patrón State - Ejemplo



```
public class ApagadoState implements State
{
    // Referencia a la clase de contexto
    private Vehiculo v;

    // Constructor que inyecta la dependencia en la clase actual
    public ApagadoState(Vehiculo v)
    {
        this.v = v;
    }

    @Override
    public void acelerar()
    {
        // Acelerar con el vehiculo apagado no sirve de mucho
        System.out.println("ERROR: El vehiculo esta apagado. Efectue el contacto para iniciar");
    }

    @Override
    public void frenar()
    {
        // Frenar con el vehiculo parado tampoco sirve de mucho...
        System.out.println("ERROR: El vehiculo esta apagado. Efectue el contacto para iniciar");
    }
}
```



```
@Override
public void contacto()
{
    // Comprobamos que el vehiculo disponga de combustible
    if (v.getCombustibleActual() > 0)
    {
        // El vehiculo arranca -> Cambio de estado
        //estado = PARADO;
        v.setEstado(new ParadoState(v));
        System.out.println("El vehiculo se encuentra ahora PARADO");
        v.setVelocidadActual(0);
    }
    else
    {
        // El vehiculo no arranca -> Sin combustible
        //estado = SIN COMBUSTIBLE
        v.setEstado(new SinCombustibleState(v));
        System.out.println("El vehiculo se encuentra sin combustible");
    }
}
```



```
@Override  
public void contacto()  
{
```

```
    // Comprobamos que el vehiculo disponga de combustible  
    if (v.getCombustibleActual() > 0)
```

```
{
```

```
    // El vehiculo arranca -> Cambio de estado  
    //estado = PARADO;
```

```
    v.setEstado(new ParadoState(v));
```

```
    System.out.println("El vehiculo se encuentra ahora PARADO");
```

```
    v.setVelocidadActual(0);
```

```
}
```

```
else
```

```
{
```

Cada método implementa la transición de estados. Para ello vuelve a setear el estado del vehículo.

```
    // El vehiculo no arranca -> SIN COMBUSTIBLE
```

```
    //estado = SIN COMBUSTIBLE
```

```
    v.setEstado(new SinCombustibleState(v));
```

```
    System.out.println("El vehiculo se encuentra sin combustible");
```

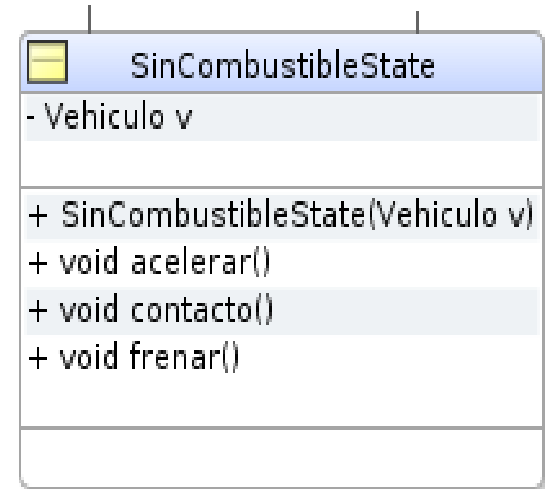
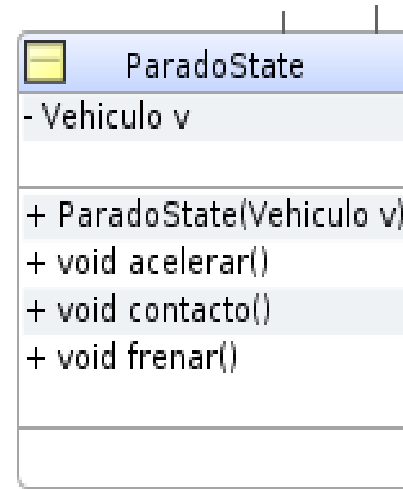
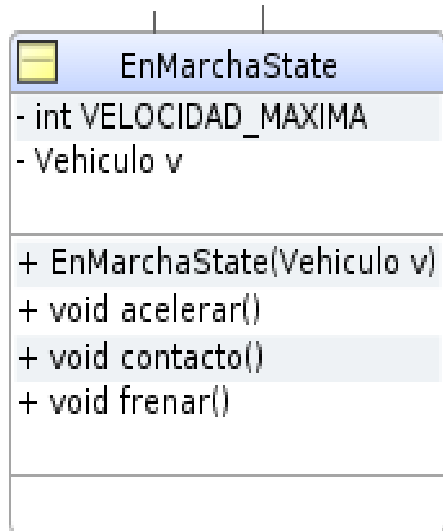
```
}
```

```
}
```

El Patrón State - Ejemplo



El resto de los estados sigue un patrón similar



El Patrón State - Ejemplo



```
public class Ejecucion
{
    public static void main(String[] args)
    {
        Vehiculo v = new Vehiculo(20);

        v.acelerar();
        v.contacto();
        v.acelerar();
        v.acelerar();
        v.acelerar();
        v.frenar();
        v.frenar();
        v.frenar();
        v.frenar();
    }
}
```

ERROR: El vehiculo esta apagado. Efectue el contacto para iniciar
Velocidad actual: 0. Combustible restante: 20
El vehiculo se encuentra ahora PARADO
El vehiculo se encuentra ahora EN MARCHA
Velocidad actual: 10. Combustible restante: 10
Velocidad actual: 20. Combustible restante: 0
El vehiculo se ha quedado sin combustible
Velocidad actual: 20. Combustible restante: 0
ERROR: El vehiculo se encuentra sin combustible
ERROR: El vehiculo se encuentra sin combustible
ERROR: El vehiculo se encuentra sin combustible
ERROR: El vehiculo se encuentra sin combustible

El Patrón State - Ejemplo



Descargar el ejemplo del repositorio GitHub: <https://github.com/leonelguccione/PatronState>

Frameworks

Reutilización de Software

- Los **objetos** normalmente **son** muy **pequeños** y están especializados para una aplicación particular, como para reutilizarlos en diferentes sistemas.
- La **reutilización** orientada a objetos tiene mejor apoyo en un proceso de desarrollo orientado a objetos **a través de** abstracciones de grano más grueso, llamadas **frameworks** (estructuras).
- Un framework es una **estructura genérica** que se extiende para crear una aplicación o un subsistema más específico.

Frameworks

. . . un conjunto integrado de artefactos de software (tales como clases, objetos y componentes), que colaboran en la facilitación de una arquitectura de reutilización para una familia de aplicaciones relacionadas.

Schmidt, 2004

Frameworks

- Los frameworks apoyan la reutilización de diseño en la que ofrecen una arquitectura que sirve de esqueleto para la aplicación, así como la reutilización de clases específicas en el sistema.
- La arquitectura se define por las clases de objetos y sus interacciones.
- Las clases se reutilizan directamente y pueden extenderse utilizando características como la herencia.

Frameworks

- Los frameworks se implementan como una **colección de clases** de objetos concretos y abstractos en un lenguaje de programación orientado a objetos. Por lo tanto, los frameworks son **específicos del lenguaje**.
- Un framework puede incorporar muchos otros frameworks, cada uno de los cuales se diseña para soportar el desarrollo de parte de la aplicación. Es posible usar un framework para crear una aplicación completa o implementar parte de una aplicación, como la interfaz de usuario gráfica.

Frameworks

- 1) Frameworks de infraestructura de sistema:** apoyan el desarrollo de infraestructuras de sistema como comunicaciones, interfaces de usuario y compiladores (Schmidt, 1997).
- 2) Frameworks de integración de middleware:** consisten en un conjunto de estándares y clases de objetos asociados que soportan comunicación de componentes e intercambio de información. Los ejemplos de este tipo de framework incluyen .NET de Microsoft y Enterprise Java Beans (EJB).
- 3) Frameworks de aplicación empresarial:** se ocupan de dominios de aplicación específicos, tales como los sistemas de telecomunicaciones o financieros (Baumer et al., 1997). El conocimiento del dominio de la aplicación integra y apoya el desarrollo de aplicaciones de usuario final.

Frameworks

WAF - frameworks de aplicación Web

(WAF) son un tipo de framework más reciente e importante.

La arquitectura de un WAF se basa por lo general en el patrón compuesto Modelo-Vista-Controlador (MVC) (Gamma et al., 1995)

A menudo, los frameworks son implementaciones de patrones de diseño. Por ejemplo, un framework MVC incluye el patrón Observer, el patrón Strategy, el patrón Composite y algunos otros que analizan Gamma y sus colaboradores (1995).

Frameworks

Características:

- Seguridad
- Páginas Web dinámicas
- Soporte de base de datos
- Gestión de sesión
- Interacción de usuarios

Frameworks

Los frameworks tienen como objetivo principal ofrecer una funcionalidad definida, auto contenida, siendo contruidos usando patrones de diseño, y su característica principal es su alta cohesión y bajo acoplamiento.

Para acceder a esa funcionalidad, se construyen piezas, objetos, llamados objetos calientes, que vinculan las necesidades del sistema con la funcionalidad que este presta. Esta funcionalidad, está constituida por objetos llamados fríos, que sufren poco o ningún cambio en la vida del framework, permitiendo la portabilidad entre distintos sistemas.

Frameworks conocidos que se pueden mencionar por ejemplo son Spring Framework, Hibernate, donde lo esencial para ser denominados frameworks es estar contruidos por objetos casi estáticos con funcionalidad definida a nivel grupo de objetos y no como parte constitutiva de estos, por ejemplo en sus métodos, en cuyo caso se habla de un API o librería.

Frameworks – características notables

- **Inversión de control**: en un frame, a diferencia de las bibliotecas, el flujo de control no es dictado por el programa que llama, sino por el mismo framework.
- **Funcionalidad o comportamiento predeterminado**: un frame tiene un comportamiento predeterminado. Este comportamiento por defecto debe ser un comportamiento útil, definido e identificable.
- **Extensibilidad**: un frame puede ser ampliado para proporcionar una funcionalidad específica. El frame, en general, no se supone que deba ser modificado, excepto en cuanto a extensibilidad. Los usuarios pueden ampliar sus características, pero no deben ni necesitan modificar su código.